

Restoration Studio: Universal, Hard-Routed and Soft Mixture-of-Experts Image Restoration, and Style-Conditioned Face-to-Sketch Generation

Afnan Rizwan

Generative AI, Assignment 1

Repository: <https://github.com/AfnanRizwan1/GenAI-A1>

Demonstration video: [\[TODO: insert the YouTube link\]](#)

Abstract—We design, train, evaluate and deploy four related generative systems and integrate them into one web application. Three of them restore 128×128 images of the Oxford-IIIT Pet dataset that are corrupted at run time by salt-and-pepper noise, Gaussian blur or rectangular occlusion: (1) a single universal denoising autoencoder with a compressed bottleneck, (2) a corruption classifier that hard-routes each image to one of three specialist autoencoders, and (3) a jointly trained soft mixture of experts in which a gate assigns a continuous weight to an identity branch and to the three experts. The fourth system is (4) a style-conditioned conditional GAN that translates face photographs of the FS2K dataset into sketches in one of three styles. Hyper-parameters of every model are tuned with Optuna, experiments are tracked with MLflow, the inference models are exported to ONNX and verified against PyTorch, and the whole system runs from one `docker compose` command behind a React and FastAPI application whose interface was designed in Google Stitch. On the fixed test manifest of 36,690 corrupted images, the mean PSNR is 23.3 dB without restoration, 24.0 dB for the universal autoencoder, 28.6 dB for hard routing (classifier accuracy 99.8 %) and 28.0 dB for the soft mixture, which has the best SSIM (0.837); the sketch generator reaches an SSIM of 0.48 and uses its style condition on every style.

Index Terms—denoising autoencoder, mixture of experts, conditional GAN, image restoration, Optuna, ONNX, FastAPI

I. INTRODUCTION

Real images are rarely clean: sensors add impulse noise, optics and motion blur the picture, and objects occlude parts of the scene. A practical restoration system therefore faces a choice. One model can be trained to cope with every degradation (a universal model), or the degradation can be recognised first and a specialist applied (hard routing). Both have drawbacks: the universal model must share capacity among very different inverse problems, whereas hard routing fails when the classifier is wrong or when an image carries more than one degradation. A differentiable *soft* mixture of experts removes the hard decision and lets the routing be learned from the reconstruction error.

This report studies this progression on a controlled benchmark (Tasks 1–3) and adds a fourth, generative task: translating face photographs into sketches whose style is chosen by the user (Task 4). The assignment is as much an engineering exercise as a modelling one, so the report also documents the data pipeline, the hyper-parameter searches, the experiment tracking, the ONNX export and the deployed application.

All source code, configuration, Optuna studies, export code, Dockerfiles and the instructions to run everything are in the repository linked on the title page.

Contributions: (i) a run-time corruption pipeline with deterministic validation and test manifests shared by all three restoration tasks; (ii) a comparison of a universal autoencoder, oracle-routed and classifier-routed specialists and a jointly fine-tuned mixture of experts under identical conditions; (iii) an analysis of the learned routing, including checks for inactive or dominating experts; (iv) a style-conditioned pix2pix-style generator and discriminator in which a learned style embedding enters both networks; (v) a containerised application that exposes all four systems and reports timing, routing and quality information.

II. RELATED WORK

Denoising autoencoders. Vincent *et al.* showed that reconstructing a clean input from a corrupted one forces an autoencoder to learn robust features [1]. Our restoration models follow this idea with a convolutional encoder, a genuinely compressed code and a convolutional decoder. We combine an L_1 term with a structural-similarity term [2], which rewards preserved edges and local contrast rather than only pixel agreement.

U-Net and skip connections. U-Net [3] concatenates encoder features into the decoder. This is excellent for translation tasks and is used in our Task 4 generator, but unrestricted skip connections would let a restoration model copy the corrupted input around the bottleneck, so the Task 1–3 autoencoders have none.

Mixtures of experts. Jacobs *et al.* introduced gated mixtures of local experts [4]; Shazeer *et al.* scaled them with a sparsely-gated layer [5], and Switch Transformers [6] added an auxiliary load-balancing loss to stop the router from collapsing onto few experts. Our soft mixture is dense (every branch contributes) and uses a balance regulariser on the batch-mean routing weights in the same spirit.

Conditional GANs and image-to-image translation. GANs [7] and their conditional variants [8] underlie pix2pix [9], which pairs a U-Net generator with a PatchGAN discriminator and an L_1 reconstruction term. We follow this recipe and make the condition (a sketch style) part of both

networks through a learned embedding, using feature-wise linear modulation [10] in the generator.

Hyper-parameter optimisation and tooling. We use Optuna [11] with the tree-structured Parzen estimator [12] and a median pruner, MLflow [13] for tracking, PyTorch [14] for modelling and ONNX Runtime [15] for inference.

III. DATASETS AND DATA PIPELINE

A. Oxford-IIIT Pet (Tasks 1–3)

We use the 3680 images of the official *trainval* split as development data and the 3669 images of the official *test* split only for final evaluation. The class labels are not used; the clean image is the restoration target. Development data are divided 80/20 with random seed 42 into 2944 training and 736 validation images, and the same split is used by all three tasks. Every image is converted to RGB and resized to 128×128 once and cached.

B. Run-time corruption

During training each clean image is turned into one of four input conditions with equal probability, and a new condition and severity are drawn every time an image is loaded; no corrupted copy is stored. The corruption is computed on the GPU on whole batches:

- **Salt-and-pepper:** a fraction $p \sim \mathcal{U}(0.02, 0.15)$ of pixels is replaced by black or white with equal probability.
- **Gaussian blur:** kernel size from $\{3, 5, 7\}$, standard deviation $\sigma \sim \mathcal{U}(0.5, 2.5)$ (separable convolution with reflect padding, equal to OpenCV’s `GaussianBlur`; tested to 10^{-4}).
- **Occlusion:** one to three black rectangles whose total area is $\mathcal{U}(10\%, 35\%)$ of the image, at random positions and with aspect ratios in $[0.5, 2]$.

The batched training version places rectangles independently, so overlapping rectangles make the achieved coverage slightly smaller than the target. The deterministic version used for the manifests avoids overlap by rejection sampling and records the achieved coverage.

C. Deterministic validation and test manifests

Validation and test corruptions are generated once and stored (type, severity level, parameters, rectangle coordinates and a per-entry random seed), so every model is evaluated on exactly the same corrupted images. The validation manifest contains, for each of the 736 validation images, the clean image and one entry per corruption type, with severity cycling through low, medium and high inside equal thirds of the training ranges (2944 entries). The test manifest contains for each of the 3669 test images the clean image and, for every corruption, the three fixed levels required by the assignment (36690 entries): salt-and-pepper $p \in \{0.03, 0.08, 0.15\}$; blur $(k, \sigma) \in \{(3, 0.7), (5, 1.5), (7, 2.5)\}$; occlusion covering about 10%, 20% and 35% with one, two and three rectangles. Replaying a manifest is deterministic, and regenerating it reproduces the committed files byte by byte.

D. FS2K (Task 4)

FS2K [16] contains 2104 photograph–sketch pairs in three sketch styles. We use the official training portion (1058 pairs) and test portion (1046 pairs). Fifteen percent of the training pairs, stratified by style with seed 42, form the validation set, which gives 899 training pairs and 159 validation pairs (54, 52 and 53 per style). The test set is style-skewed (619, 381 and 46 images for styles 1, 2 and 3) and is never used for training or tuning. Photographs and sketches are resized to 128×128 and the pairing is preserved by index. Augmentation (horizontal flip, rotation up to 5° , zoom 0.9–1.0, shift up to 5%) is applied by a single affine warp to the six-channel stack of photograph and sketch, so the pixel correspondence is preserved by construction.

IV. TASK 1: UNIVERSAL MULTI-CORRUPTION DENOISING AUTOENCODER

A. Architecture

The encoder has four stages, each a stride-2 4×4 convolution followed by a 3×3 convolution (batch normalisation [17] and LeakyReLU after each), taking the spatial size from 128 to 8 while the channels grow from c to $8c$. The bottleneck is one of two kinds, chosen by Optuna: a *linear* code (flatten, then a fully connected layer to d numbers) or a *convolutional* code (a 1×1 convolution to c_z channels, i.e. a $c_z \times 8 \times 8$ tensor of $64c_z$ numbers). Dropout is applied to the code. The decoder mirrors the encoder with transposed convolutions and ends with a sigmoid. Even the smallest code is more than ten times smaller than the 49152 input values. There are *no skip connections*, so every output pixel depends only on the code (a unit test checks that decoding the encoder output reproduces the forward pass). Because no skip connection is used there is nothing to justify or ablate in the sense of the assignment’s remark on limited skips; a skip-connection ablation was not run.

B. Loss, optimiser and tuning

With x the clean image, $\tilde{x}$ the corrupted input and $\hat{x} = D(E(\tilde{x}))$ the reconstruction,

$$\mathcal{L} = \alpha \|\hat{x} - x\|_1 + (1 - \alpha)(1 - \text{SSIM}(\hat{x}, x)), \quad (1)$$

with SSIM computed over 11×11 Gaussian windows ($\sigma = 1.5$). Models are trained with AdamW [18], a one-cycle learning-rate schedule [19], mixed precision, and the whole dataset resident on the GPU. The Optuna study (TPE sampler, seed 42, median pruner) searches the learning rate $[3 \cdot 10^{-4}, 3 \cdot 10^{-3}]$ (log), batch size $\{32, 64, 128\}$, base channels $c \in \{16, 32, 48, 64\}$, dropout $[0, 0.3]$, the loss weight $\alpha \in [0.5, 0.95]$, the bottleneck type, and its size ($d \in \{128, 256, 512, 1024\}$ or $c_z \in \{4, 8, 16, 32\}$). The objective, minimised on the validation manifest, is $L_1 + (1 - \text{SSIM})$, which is independent of α so that trials with different α are comparable. The planned budget is 30 trials of 10 epochs followed by 150 epochs of final training. The study completed 30 trials: 13 ran to the end and 17 were pruned. The best trial

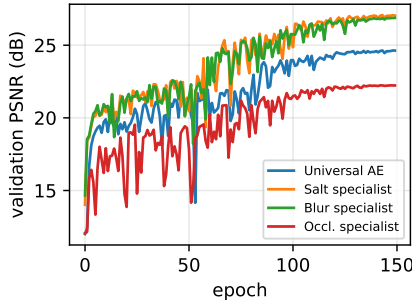
(number 15, validation objective 0.414 after 10 epochs) used the convolutional bottleneck with $c_z = 32$, $c = 48$, learning rate $7.7 \cdot 10^{-4}$, batch size 32, dropout 0.10 and $\alpha = 0.51$, and all 13 completed trials used the convolutional bottleneck. Two selected values lie at the edge of the allowed range ($c_z = 32$ is the largest code that was searched and $\alpha = 0.51$ is close to the smallest weight, 0.5), so a larger code or an even stronger SSIM weight might have scored better. The final model, trained for 150 epochs, reached a validation PSNR of about 24.6dB and SSIM of 0.79 (all four input conditions averaged).

C. A first run and the bottleneck study

A first, smaller run (8 trials of 5 epochs, linear bottleneck only, 40 final epochs) reached a validation PSNR of 18.75dB and SSIM of 0.471, and the curve had flattened. Because the linear code discards spatial layout, we added the convolutional bottleneck to the search space for the main run. The main run reached a validation PSNR of 24.6dB and SSIM of 0.79, that is +5.9dB and +0.32 over the first run, and Optuna chose the convolutional bottleneck in every completed trial. The gain combines the code that keeps the 8×8 spatial layout (2048 numbers for $c_z = 32$ instead of one vector) with the larger search and the longer training; we did not separate these effects.

D. Results

Tables V and VI report PSNR and SSIM on the fixed test manifest for every corruption and severity, together with the no-restoration baseline (the corrupted input itself), which every model must beat.



Against the no-restoration baseline the universal autoencoder helps most where the damage is largest. For salt-and-pepper noise PSNR rises from 20.1, 15.9 and 13.1 dB (low, medium, high severity) to 25.3, 25.2 and 25.1 dB, and for occlusion from 16.7, 13.3 and 10.8dB to 23.1, 21.6 and 19.8dB. Its output quality is almost independent of the input condition (about 25 dB for clean, salt-and-pepper and mildly blurred images). This shows that the quality is limited by the reconstruction ceiling of the bottleneck and not by the corruption: even a *clean* image comes back at only 25.3dB (SSIM 0.82). Consequently the model does not beat the input where the input is already close to the target: clean images (60dB for the identity mapping), low and medium

blur (32.3 and 26.8 dB for the input against 25.4 and 25.3 dB) and, in SSIM, low occlusion (0.862 against 0.771). Only for high blur do the two meet in PSNR (24.4 against 24.2dB), and there the autoencoder is better in SSIM (0.740 against 0.692). This is the price of a strict bottleneck without skip connections: the model removes noise and fills in occlusions, but it cannot reproduce fine detail.

The failure cases in Fig. 2 show two different problems. First, the colour is not reproduced faithfully: the pink background of the salt-and-pepper example comes out beige and the purple background of the blur example blue-grey, so the PSNR (15.8dB) is much lower than that of the input (20.8 and 37.0dB). The strict bottleneck keeps the shape of the animal but not the exact colour of large uniform regions, and these regions dominate the pixel error. Second, in the two occlusion examples the animal sits on a black background, so a black rectangle is almost invisible and the input is already close to the target (58.2 and 48.9 dB). The autoencoder nevertheless reconstructs a brown glow in the upper right corner, so restoration makes these images much worse (13.1 and 12.6dB). Both effects come from the same cause: the network always reconstructs from a compressed code and cannot copy pixels that were fine.

V. TASK 2: CORRUPTION CLASSIFICATION AND HARD-ROUTED SPECIALISTS

A. Classifier

The classifier has four blocks of a stride-1 and a stride-2 3×3 convolution (batch normalisation, ReLU), global average pooling, dropout and a linear layer with four outputs. Its labels come from the run-time corruption pipeline, and every training batch is exactly balanced (equal numbers of clean, salt-and-pepper, blur and occlusion samples), so the classifier cannot favour one class. It is trained with cross-entropy and AdamW. The Optuna study searches the learning rate, batch size, channel configuration (small, medium, large), dropout and weight decay, and maximises validation macro-F1. The study completed 30 trials (6 ran to the end, 24 were pruned). The best trial, number 21, validation macro-F1 0.9955 after 10 epochs) used the small channel configuration, learning rate $3.7 \cdot 10^{-3}$, batch size 32, dropout 0.44 and weight decay $9.5 \cdot 10^{-5}$; the dropout lies near the upper end of the searched range (0.5). The final model, trained for 60 epochs, reached a validation macro-F1 of 0.9956.

B. Specialists and hard routing

Three specialist autoencoders share one architecture but are trained independently, each only on its own corruption and with the same clean targets. To keep the cost feasible the architecture is chosen by one shared Optuna search on the three corruptions (clean images excluded) with the same space and objective as Task 1; the three specialists are then trained from scratch with that configuration. At inference the classifier's arg-max selects the specialist, and a clean prediction uses an identity bypass (no expert is called).

TABLE I
TASK 2 CLASSIFIER ON THE TEST MANIFEST (PER CLASS, MACRO AVERAGE).

Class	Precision	Recall	F1	Support
Clean	0.984	0.996	0.990	3669
Salt-and-pepper	1.000	1.000	1.000	11007
Gaussian blur	0.999	1.000	1.000	11007
Occlusion	0.999	0.994	0.997	11007
Macro average	0.996	0.998	0.996	36690

TABLE II
CLASSIFIER ACCURACY BY CORRUPTION AND SEVERITY (%).

Corruption	Level	Accuracy (%)
Clean	–	99.6
Salt-and-pepper	low	100.0
Salt-and-pepper	medium	100.0
Salt-and-pepper	high	100.0
Gaussian blur	low	100.0
Gaussian blur	medium	100.0
Gaussian blur	high	100.0
Occlusion	low	98.3
Occlusion	medium	100.0
Occlusion	high	100.0

C. Evaluation modes

Oracle routing selects the expert from the true label in the test manifest and measures the specialists in isolation. *Predicted routing* uses the classifier and measures the operational system. The difference between the two is the cost of classifier errors.

D. Results

Test accuracy of the classifier is 99.79% (Table I, Fig. 3); 78 of the test entries were misrouted. Corruption type is recognised almost perfectly: all salt-and-pepper and blur entries are classified correctly at every severity. The remaining errors are between clean and occlusion. Clean images have the lowest precision (0.984) because some occluded images are taken for clean ones, and occlusion has the lowest recall (0.994); the only severity below 99.6% is low occlusion (98.3%), where a single small rectangle is easy to miss, particularly on a dark background.

With oracle routing the specialists are better than the universal autoencoder in every corrupted condition, by 0.8 to 1.6 dB PSNR (for example 26.8 against 25.3 dB for low salt-and-pepper noise and 25.8 against 24.2 dB for high blur) and by about 0.01 to 0.04 SSIM, which supports the idea that three narrower problems are easier than one broad one. They share the limitation of the bottleneck: for low and medium blur they remain below the uncorrected input (26.4 and 26.6 dB against 32.3 and 26.8 dB). Predicted routing is practically as good as oracle routing. For all nine corrupted conditions the two PSNR values agree to within 0.01 dB; the only visible effect of classifier errors is on clean images, where the mean PSNR falls from 60 (the cap for an exact identity) to 59.85 dB

because some clean inputs were sent through an expert. On this benchmark the cost of the classifier’s mistakes is therefore negligible. The four misrouted examples in Fig. 4 are all clean photographs with wide black borders (pictures that were letterboxed before the dataset was assembled). The classifier takes these borders for occlusion rectangles and sends the image to the occlusion specialist, which rebuilds it at 14.8 to 17.8 dB instead of leaving it untouched (60 dB). Such errors are rare, but a single one is costly because the identity bypass is lost.

VI. TASK 3: JOINTLY TRAINED SOFT MIXTURE OF EXPERTS

A. Model

Let $z(\tilde{x}) \in \mathbb{R}^4$ be the logits of the gate and E_s, E_b, E_o the salt-and-pepper, blur and occlusion experts. With temperature T ,

$$g = \text{softmax}(z(\tilde{x})/T) = [g_0, g_s, g_b, g_o], \quad (2)$$

$$\hat{x} = g_0 \tilde{x} + g_s E_s(\tilde{x}) + g_b E_b(\tilde{x}) + g_o E_o(\tilde{x}). \quad (3)$$

The first branch is an identity branch for clean inputs. The combination is differentiable, so gate and experts are trained through the final reconstruction error. The gate is initialised from the Task 2 classifier and the experts from the three trained Task 2 specialists; nothing starts from random weights.

B. Training

Training has two stages: a warm-up in which the experts are frozen (and kept in evaluation mode, so their batch-norm statistics do not change) and only the gate is trained, followed by joint fine-tuning of all components with a smaller learning rate. The loss is

$$\mathcal{L} = a \mathcal{L}_1 + (1 - a)(1 - \text{SSIM}) + \gamma \text{CE}(\frac{z}{T}, c) + \delta \mathcal{L}_{\text{bal}}, \quad (4)$$

where c is the true corruption class and

$$\mathcal{L}_{\text{bal}} = \sum_{i=1}^K (\bar{g}_i - \frac{1}{K})^2, \quad K = 4, \quad (5)$$

with $\bar{g}_i$ the mean weight of branch i over a class-balanced batch, so the target for a healthy gate is uniform usage of the four branches over the batch. The assignment’s starting weights are $a = 0.8$ ($\beta = 0.2$ for SSIM, here $1 - a$), $\gamma = 0.1$ and $\delta = 0.01$. (The assignment text gives the balance formula only in a garbled form, so we use the squared deviation from uniform usage, the simplest differentiable regulariser of this kind; Switch Transformers [6] use a related load-balancing loss.) Optuna tunes the joint learning rate $[10^{-5}, 5 \cdot 10^{-4}]$ (log), temperature $T \in [0.5, 3]$, classification weight $\gamma \in [0.01, 1]$ (log), balance weight $\delta \in [10^{-3}, 1]$ (log) and $a \in [0.5, 0.95]$. A trial is pruned when its validation objective is poor, or when one branch receives a mean weight above 0.9 after the warm-up (routing collapse). All 20 trials completed and none was pruned; no trial collapsed. The best trial (number 15, validation objective 0.166) used joint learning rate $2.0 \cdot 10^{-4}$, temperature $T = 2.18$, $\gamma = 0.012$, $\delta = 0.050$ and $a = 0.53$; γ and a lie

TABLE III
TASK 3: MEAN ROUTING WEIGHTS ON THE TEST MANIFEST BY TRUE CORRUPTION AND SEVERITY (LARGEST PER ROW IN BOLD).

True corruption	Level	Identity	Salt	Blur	Occl.
Clean	–	0.942	0.001	0.008	0.048
Salt-and-pepper	low	0.079	0.898	0.010	0.013
Salt-and-pepper	medium	0.013	0.983	0.002	0.002
Salt-and-pepper	high	0.005	0.994	0.001	0.001
Gaussian blur	low	0.614	0.001	0.378	0.007
Gaussian blur	medium	0.181	0.000	0.818	0.001
Gaussian blur	high	0.049	0.000	0.950	0.001
Occlusion	low	0.595	0.001	0.005	0.398
Occlusion	medium	0.212	0.001	0.001	0.786
Occlusion	high	0.044	0.000	0.000	0.955

TABLE IV
TASK 3: EXPERT ACTIVITY CHECKS OVER ALL TEST ENTRIES.

Branch	Mean w	Own	Other	Argmax (%)	Inactive	Dominates
identity	0.273	0.942	0.199	26.7	no	no
salt	0.288	0.958	0.001	29.7	no	no
blur	0.217	0.715	0.004	21.6	no	no
occlusion	0.221	0.713	0.010	22.0	no	no

close to the lower edge of their ranges. The final model was trained for 40 epochs after a warm-up of 3 epochs.

C. Routing analysis

83.25% of test entries have their largest weight on the branch that matches the true condition. Table III lists the mean routing weights per true corruption and severity and Fig. 5 shows them as a heat map. Table IV checks for an inactive expert (mean weight below 0.05 over all inputs) and for an expert that dominates unrelated inputs (mean weight above 0.5 on inputs of other corruption types). The gate learned a sensible division of labour. Salt-and-pepper noise goes to the salt expert at every severity (0.90 to 0.99), blur to the blur expert and occlusion to the occlusion expert, and the weight of the matching expert grows with severity (blur 0.38, 0.82, 0.95; occlusion 0.40, 0.79, 0.96). For clean images the identity branch receives 0.94. At low blur and low occlusion the identity branch gets the larger share (0.61 and 0.60), which is reasonable because these inputs are barely damaged; it also explains why the argmax matches the true condition for only 83.3% of entries, since the gate is not trained to name the corruption but to restore. No expert is inactive (mean weights between 0.22 and 0.29) and none dominates unrelated inputs (mean weight on other types at most 0.010 for the experts; the identity branch has 0.20 on corrupted inputs because of the mild cases above). The examples in Fig. 6 show both regimes: confident routing has an entropy of 0.0004 nats, while the spread-out cases reach 1.10 nats (maximum $\ln 4 = 1.39$). Compared with hard routing, the soft system has a lower mean L_1 error (0.0323 against 0.0343) and a higher SSIM in every corrupted condition (overall 0.837 against 0.811), and it is clearly better on low blur (30.4 against 26.5 dB), where the identity branch keeps the sharp input. It is worse in PSNR

on clean images (52.4 against 60 dB, since the weight on the identity branch is 0.94 rather than 1) and on low and medium occlusion (21.3 against 24.1 dB and 22.1 against 22.5 dB), so its overall PSNR is lower (28.0 against 28.6 dB). Hard routing is therefore better when the classifier is right and the cost of mixing matters, and soft routing gives smoother behaviour for mild corruptions; neither dominates.

VII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH GENERATION

A. Architecture

Generator. A U-Net with six down-sampling and six up-sampling levels ($128 \rightarrow 2 \rightarrow 128$) and skip connections. The style $s \in \{1, 2, 3\}$ selects a learned embedding $e_s \in \mathbb{R}^m$ (m tuned). The embedding enters the generator in two places, so it is part of the model and not an interface label: it is broadcast over the image and concatenated to the input photograph, and in every decoder block it produces a per-channel scale and shift (FiLM [10]) applied after instance normalisation [20]. **Discriminator.** A PatchGAN that receives the photograph, the (real or generated) sketch and the broadcast style embedding and outputs a grid of real/fake logits (14×14 for 128×128 inputs). A unit test confirms that the style embedding receives gradients in both networks.

B. Objective and training

With photograph x , style s and target sketch y , the discriminator minimises the average of the binary cross-entropy on real pairs (x, s, y) and fake pairs $(x, s, G(x, s))$ (logged separately as D-real and D-fake) and the generator minimises

$$\mathcal{L}_G = \text{BCE}(D(x, s, G(x, s)), 1) + \lambda \|G(x, s) - y\|_1, \quad (6)$$

with adversarial and reconstruction parts logged separately. Both networks use Adam ($\beta = (0.5, 0.999)$). Optuna searches the generator and discriminator learning rates $[5 \cdot 10^{-5}, 10^{-3}]$ (log), batch size $\{8, 16, 32\}$, base channels $\{16, 32, 48, 64\}$, dropout $[0, 0.5]$, embedding dimension $\{8, 16, 32, 64\}$ and $\lambda \in [10, 200]$ (log), using 15 epochs per trial; the best configuration is retrained for 150 epochs. The objective is validation $L_1 + (1 - \text{SSIM})$. The same first eight validation photographs are rendered every ten epochs to follow the generator’s development.

C. Results

Table VII reports L_1 , SSIM and PSNR on the official test split per style, together with a baseline that simply converts the photograph to grayscale. Table VIII checks style conditioning: every test photograph is generated with all three styles and compared with its ground-truth sketch. On the 1,046 test pairs the generator reaches an L_1 error of 0.106, an SSIM of 0.481 and a PSNR of 15.6 dB, against 0.410, 0.268 and 6.8 dB for the grayscale photograph without any generator. Style 2 is the hardest (12.4 dB, SSIM 0.398), which fits the visual impression: its targets have heavy dark strokes and dense hatching that cannot be matched pixel by pixel. Style 1 (thin lines) and style 3 reach 17.4 and 18.7 dB, but style 3 has

TABLE V

RESTORATION QUALITY ON THE FIXED TEST MANIFEST: PSNR (dB). THE FIRST COLUMN IS THE CORRUPTED INPUT WITHOUT RESTORATION. BEST PER ROW IN BOLD (CLEAN IMAGES GIVE 60 dB FOR AN IDENTITY MAPPING, THE CAP USED FOR IDENTICAL IMAGES).

Corruption	Level	No restoration	Universal AE	Hard (oracle)	Hard (predicted)	Soft MoE
Clean	–	60.00	25.28	60.00	59.85	52.38
Salt-and-pepper	low	20.14	25.27	26.80	26.80	27.22
Salt-and-pepper	medium	15.88	25.21	26.77	26.77	26.80
Salt-and-pepper	high	13.14	25.07	26.68	26.68	26.63
Gaussian blur	low	32.34	25.38	26.45	26.45	30.39
Gaussian blur	medium	26.77	25.32	26.57	26.57	27.03
Gaussian blur	high	24.35	24.17	25.77	25.77	25.86
Occlusion	low	16.68	23.11	24.13	24.13	21.33
Occlusion	medium	13.28	21.56	22.46	22.46	22.11
Occlusion	high	10.78	19.81	20.62	20.62	20.09

TABLE VI

RESTORATION QUALITY ON THE FIXED TEST MANIFEST: SSIM. BEST PER ROW IN BOLD.

Corruption	Level	No restoration	Universal AE	Hard (oracle)	Hard (predicted)	Soft MoE
Clean	–	1.000	0.817	1.000	0.999	0.999
Salt-and-pepper	low	0.603	0.816	0.831	0.831	0.848
Salt-and-pepper	medium	0.340	0.812	0.830	0.830	0.834
Salt-and-pepper	high	0.204	0.803	0.827	0.827	0.828
Gaussian blur	low	0.944	0.815	0.824	0.824	0.918
Gaussian blur	medium	0.806	0.811	0.823	0.823	0.833
Gaussian blur	high	0.692	0.740	0.777	0.777	0.778
Occlusion	low	0.862	0.771	0.784	0.786	0.877
Occlusion	medium	0.726	0.724	0.740	0.740	0.780
Occlusion	high	0.542	0.654	0.673	0.673	0.675

TABLE VII

TASK 4 ON THE OFFICIAL FS2K TEST SPLIT (GENERATOR VS. GRAYSCALE-PHOTO BASELINE, SUBSCRIPT g).

Style	N	L1	SSIM	PSNR	$L1_g$	$SSIM_g$	$PSNR_g$
Style 1	619	0.0798	0.5212	17.36	0.4494	0.2811	6.17
Style 2	381	0.1527	0.3979	12.37	0.3517	0.2294	7.71
Style 3	46	0.0644	0.6262	18.73	0.3647	0.4144	8.18
All	1046	0.1057	0.4809	15.60	0.4101	0.2681	6.82

TABLE VIII

STYLE CONDITIONING: MEAN L_1 TO THE GROUND-TRUTH SKETCH (ROWS: TRUE STYLE, COLUMNS: STYLE GIVEN TO THE GENERATOR; LOWEST PER ROW IN BOLD).

	Fed Style 1	Fed Style 2	Fed Style 3
True Style 1	0.0798	0.1205	0.1198
True Style 2	0.1984	0.1527	0.1700
True Style 3	0.1225	0.1015	0.0644

only 46 test images, so its numbers are uncertain. The style condition is used. In Table VIII the lowest error in every row is on the diagonal, that is, each style is closest to its own target when the matching style is given (for example 0.080 against 0.120 and 0.120 for style 1). In Fig. 7 the three columns differ clearly: style 1 gives thin light lines, style 2 heavy dark strokes with shaded hair and clothes, and style 3 an intermediate look with a grey background tone. The failure cases show the limits of the model. Face outlines and eyes are placed correctly, but

TABLE IX

OPTUNA STUDIES: NUMBER OF TRIALS AND BEST OBJECTIVE VALUE (VALIDATION $L_1 + (1 - SSIM)$) FOR THE AUTOENCODERS AND THE GAN, MACRO-F1 FOR THE CLASSIFIER; LOWER IS BETTER EXCEPT FOR THE CLASSIFIER).

Study	Trials	Complete	Pruned	Best value
Task 1 autoencoder	30	13	17	0.4143
Task 2 classifier	30	6	24	0.9915
Task 2 specialists (shared)	30	10	20	0.4205
Task 3 soft MoE	20	20	0	0.1661
Task 4 cGAN	20	14	6	0.6099

hair is rendered as a smooth mass or as short scribbles rather than the long strokes of the artist, and the contrast is lower than in the targets. This is what one expects from an objective dominated by L_1 with only 899 training pairs: the generator averages over plausible strokes, and the adversarial term is too weak to sharpen them fully.

VIII. HYPER-PARAMETER OPTIMISATION

Every task uses an Optuna study (SQLite storage, TPE sampler with seed 42, median pruner, resumable) whose trials, parameters and intermediate values are also logged to MLflow. Table IX summarises the studies and lists the selected parameters.

- **Task 1 autoencoder:** lr=0.000774, batch_size=32, base_ch=48, dropout=0.1006, alpha=0.5137, bottleneck=conv, latent_ch=32

- **Task 2 classifier:** lr=0.003663, batch_size=32, channels=small, dropout=0.4426, weight_decay=9.517e-05
- **Task 2 specialists (shared):** lr=0.001367, batch_size=32, base_ch=48, dropout=0.04377, alpha=0.6252, bottleneck=conv, latent_ch=32
- **Task 3 soft MoE:** lr_joint=0.0002043, temperature=2.175, gamma=0.01208, delta=0.0498, a=0.5275
- **Task 4 cGAN:** lr_g=0.0009958, lr_d=0.0003826, batch_size=8, base_ch=48, dropout=0.1291, emb_dim=8, lam=196.2

IX. APPLICATION AND DEPLOYMENT

A. Design

The interface was designed in Google Stitch [21] before it was implemented (project: <https://stitch.withgoogle.com/projects/10012210425231381214>). The first generation (Fig. 10) already fixed the layout (sidebar with four workspaces, controls card, input / output panels, metric cards), but it contained details the system does not have (hardware names, network architectures, extra sketch controls, scores). We therefore wrote a design system (design/DESIGN.md: palette, typography, spacing, components, motion and strict rules about which data may be shown) and generated a refined design from it (Fig. 11). The React and Tailwind implementation follows the second design and shows only quantities the application really computes.

B. Architecture

The frontend (React, Tailwind CSS, served by nginx) talks to a FastAPI backend that exposes GET /api/health, POST /api/universal, /api/hard, /api/soft and /api/sketch. The backend validates every upload (content type, size, pixel count, decodability, minimum size), resizes it to 128×128 , optionally applies a chosen corruption (type, severity or custom parameters, seed) using exactly the corruption code that was used for training, runs the ONNX models with ONNX Runtime on the CPU and returns the images together with the corruption settings, inference time, PSNR and SSIM against the clean reference, and the classifier probabilities or routing weights (and their entropy). Already corrupted images can be uploaded and restored as they are. A difference map, a before/after slider and JSON export of the metrics are available.

pending figure: app'screenshots.png

[TODO: insert screenshots of the running application with the trained models (one per workspace) and replace this placeholder]

C. ONNX export and verification

All inference models (universal autoencoder, classifier with softmax, the three specialists, the complete soft mixture as one graph returning the image and the four weights, and only the generator of Task 4) are exported with opset 17 and a dynamic batch axis. Each exported model is run with ONNX Runtime and compared with PyTorch on several inputs and two batch sizes; the export command fails if the maximum absolute difference exceeds 10^{-4} (Table X).

TABLE X
ONNX EXPORT VERIFICATION (ONNX RUNTIME VS. PYTORCH).

[pending: onnx]

D. Containers, tracking and reproducibility

`docker compose up --build` builds a backend image (Python, ONNX Runtime, no PyTorch) and a frontend image (multi-stage Node build served by nginx, which also proxies /api); the trained models are mounted from `./models`. The Compose setup was verified from a fresh clone on Windows with Docker Engine running in WSL 2. All runs were logged to MLflow (parameters, losses, metrics, checkpoints and sample images). The repository contains 59 Python tests and 33 frontend tests covering the corruption pipeline, models, training loops, Optuna objectives, evaluation, ONNX export, the API and the user interface. Trained model files are larger than GitHub's file limit and are distributed as downloadable assets (`scripts/download_models.sh`).

X. LIMITATIONS

The hyper-parameter searches are small (tens of trials of a few epochs) because of the time available, so the selected configurations are good rather than optimal; for the Task 4 search the best trial lay at the upper end of the allowed range for λ . All models work at 128×128 . The strict bottleneck of the restoration autoencoders limits reconstruction detail, so for mild corruptions the corrupted input itself can be closer to the clean image than the restored one (see the baseline column in Tables V–VI). Training corruptions place occlusion rectangles independently, whereas the test manifest avoids overlap. The mixture of experts handles combined corruptions only to the extent its gate generalises; we did not construct a benchmark of images with several simultaneous corruptions. The sketch generator is trained on 899 pairs with an L_1 -dominated objective and the style-3 test set has only 46 images, so per-style numbers must be read with care. No skip-connection ablation and no repeated runs with different seeds were performed, so no confidence intervals are given.

XI. CONCLUSION

Specialisation helped. Three specialist autoencoders chosen by a classifier that is right in 99.8 % of cases beat the single universal autoencoder in every corrupted condition by 0.8 to 1.6 dB, and the cost of the classifier's rare mistakes was negligible except for clean images with black borders, which it mistook for occlusion. The jointly trained soft mixture learned a clear division of labour without any expert becoming inactive or dominant, gave the best SSIM and the best result for low blur, and kept mild corruptions close to the input, but it lost PSNR on clean images and on occlusion. For salt-and-pepper noise all three restorers clearly improved the input; for blur and for mild occlusion the strict bottleneck often removes more detail than the corruption does, so doing nothing, or

the identity branch of the mixture, can be better. The style-conditioned GAN follows its style input (the matching style has the lowest error in every case) and clearly beats a grayscale baseline, but it produces smoother and lighter strokes than the artists, especially for hair, and the style-3 result rests on only 46 test images. The application brings the four models together in one interface that starts from a single `docker compose` command. The main limits are the single training run per model without confidence intervals, hyper-parameters that sit at the edge of some searched ranges, and the missing comparison with a network with skip connections, which would show how much of the loss comes from the strict bottleneck.

APPENDIX A USE OF AI TOOLS

- **Claude Code (Anthropic, [22])** was used throughout as a coding assistant: to write and refactor the data pipeline, models, training and evaluation scripts, Optuna studies, ONNX export, the FastAPI backend, the React frontend, the Dockerfiles, the tests, the report skeleton and the figure and table generation scripts; and to discuss design decisions (for example the convolutional bottleneck, the balance regulariser, the FiLM conditioning).
- **Google Stitch [21]** generated the interface mock-ups from prompts and a design-system file written for the project; the generated designs were corrected by hand where they contained invented content.

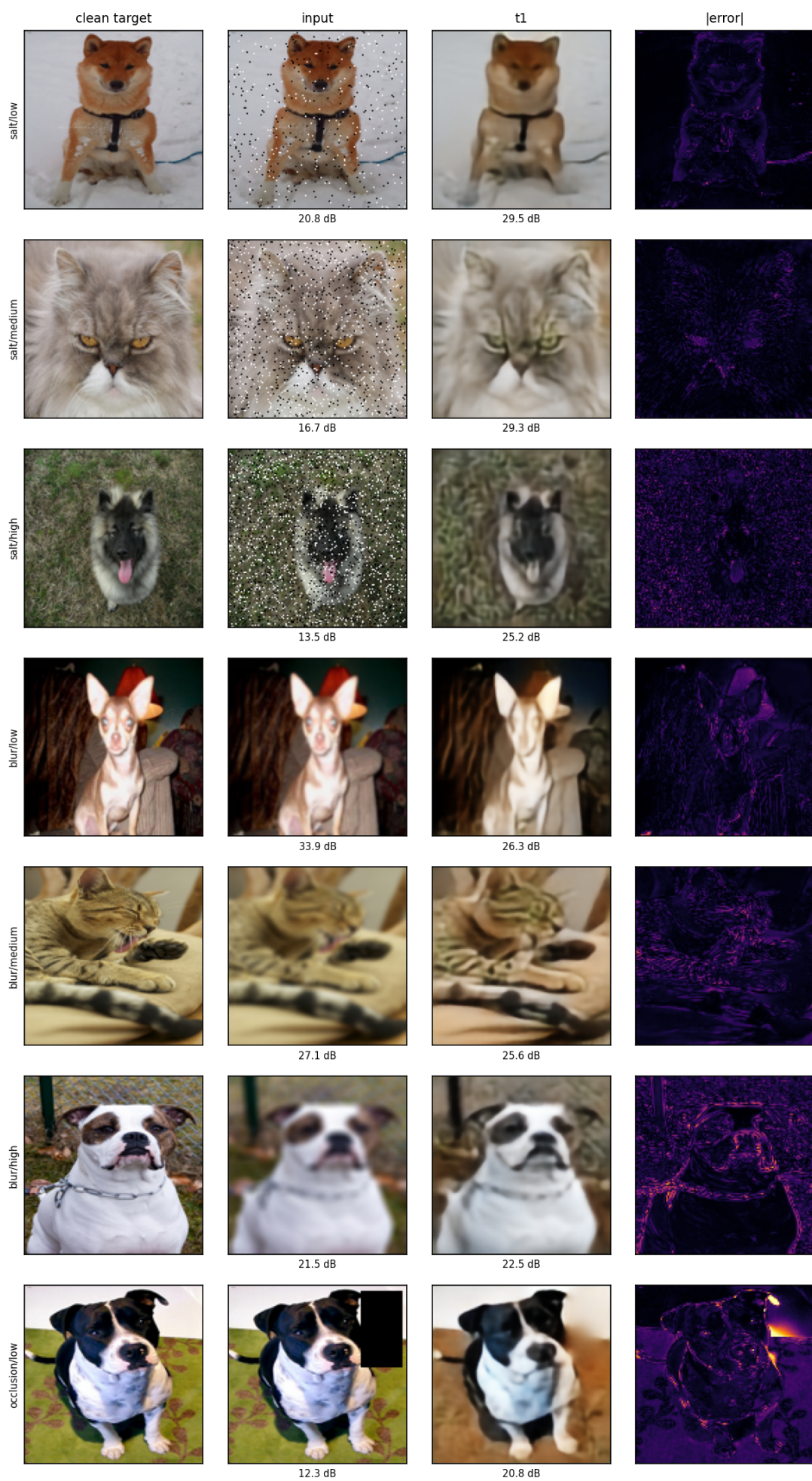
How the outputs were tested and corrected. Generated code was run, not trusted: the repository has 59 Python and 33 frontend tests, including tests that compare the batched GPU blur with OpenCV, check that decoding the code reproduces the autoencoder output (no skip path), verify that experts are frozen during warm-up, check that the augmentation transforms photograph and sketch identically, compare ONNX with PyTorch outputs, and check that the backend SSIM equals the library definition used in training. Several failures found this way were fixed (for example a wrongly ignored `src/data` folder that broke the cloud runs, a metric key naming slip in the routing analysis, and layout problems found by driving the application in a real browser), and a mutation check confirmed that the frontend tests fail when the code is deliberately broken. Training was run by the author on Kaggle GPUs, and every result in this report is read from the files written by the evaluation scripts. The student remains responsible for the submitted system. **[TODO: review and edit this appendix so it describes your own contribution accurately]**

APPENDIX B REPRODUCING THE RESULTS

- 1) `bash scripts/run_t1_t2.sh` (Task 1 search and training, classifier, specialists), `bash scripts/run_t3.sh`, `bash scripts/run_t4.sh`.
- 2) `python -m src.eval.evaluate ...` and `python -m src.eval.evaluate_t4 ...` (test-set evaluation), `python -m src.export.export_onnx ...` (ONNX).
- 3) `python scripts/make_report_assets.py` regenerates every table and figure of this report from the outputs.
- 4) `docker compose up --build` starts the application at `http://localhost:8080`.

REFERENCES

- [1] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proc. International Conference on Machine Learning (ICML)*, 2008, pp. 1096–1103.
- [2] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [3] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [4] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [5] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [6] W. Fedus, B. Zoph, and N. Shazeer, “Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.
- [7] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 2672–2680.
- [8] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [9] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 1125–1134.
- [10] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [11] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [12] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyperparameter optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2011, pp. 2546–2554.
- [13] MLflow contributors, “MLflow: A platform for the machine learning lifecycle,” <https://mlflow.org>, 2018.
- [14] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [15] ONNX Runtime developers, “ONNX Runtime: cross-platform, high performance ml inferencing and training accelerator,” <https://onnxruntime.ai>, 2021.
- [16] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin, and L. V. Gool, “Facial-sketch synthesis: A new challenge,” *Machine Intelligence Research*, vol. 19, pp. 257–287, 2022.
- [17] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [18] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [19] L. N. Smith and N. Topin, “Super-convergence: Very fast training of neural networks using large learning rates,” in *Proc. SPIE Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications*, 2019.
- [20] D. Ulyanov, A. Vedaldi, and V. Lempitsky, “Instance normalization: The missing ingredient for fast stylization,” *arXiv preprint arXiv:1607.08022*, 2016.
- [21] Google, “Stitch: Ai-assisted user interface design,” <https://stitch.withgoogle.com>, 2025.
- [22] Anthropic, “Claude code: an agentic coding assistant,” <https://www.anthropic.com>, 2025.



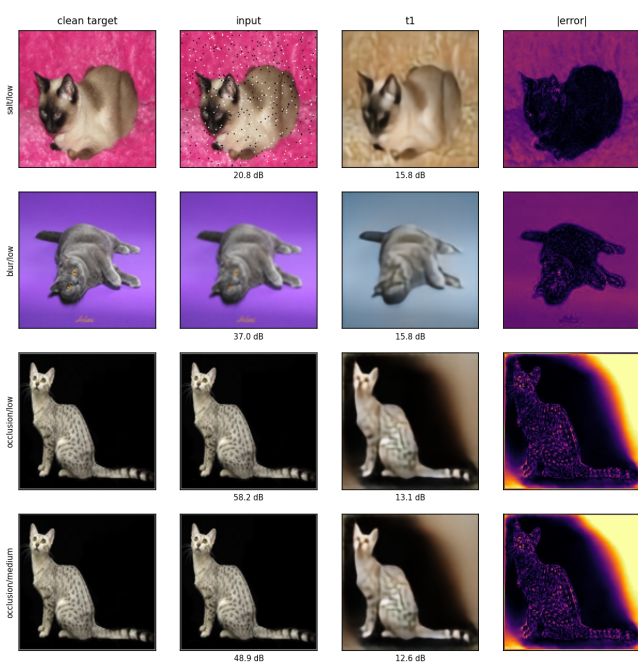


Fig. 2. Task 1 failure cases: for each corruption the test entry with the lowest PSNR gain over the input, and the worst remaining entry.

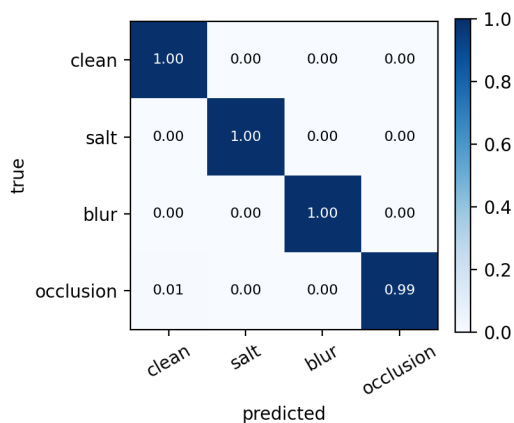


Fig. 3. Row-normalised confusion matrix of the four-class classifier on the test manifest.

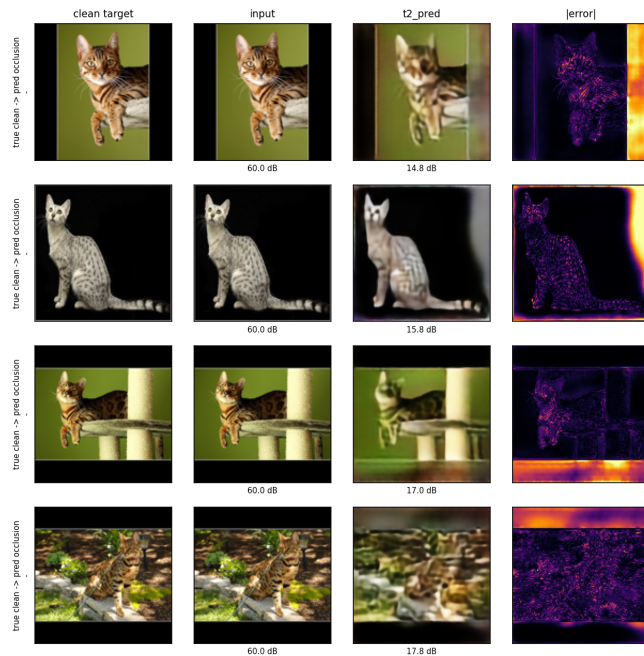


Fig. 4. Misrouting cases: the four classifier errors with the largest PSNR loss compared with oracle routing.

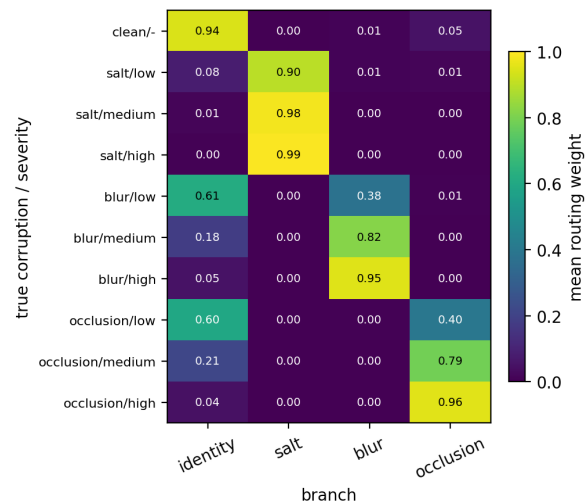


Fig. 5. Mean routing weights by true corruption and severity.

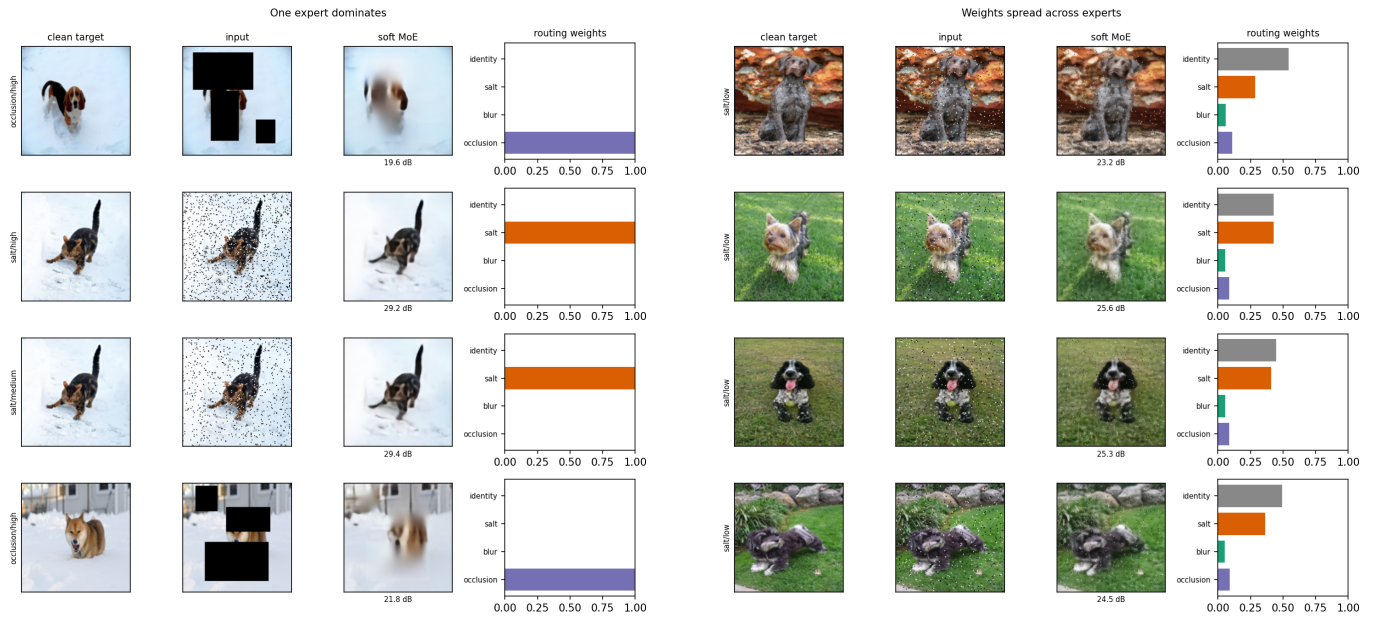


Fig. 6. Left: inputs for which one expert dominates (lowest routing entropy). Right: inputs for which the weights are spread over several branches (highest routing entropy). Bars show the four routing weights.

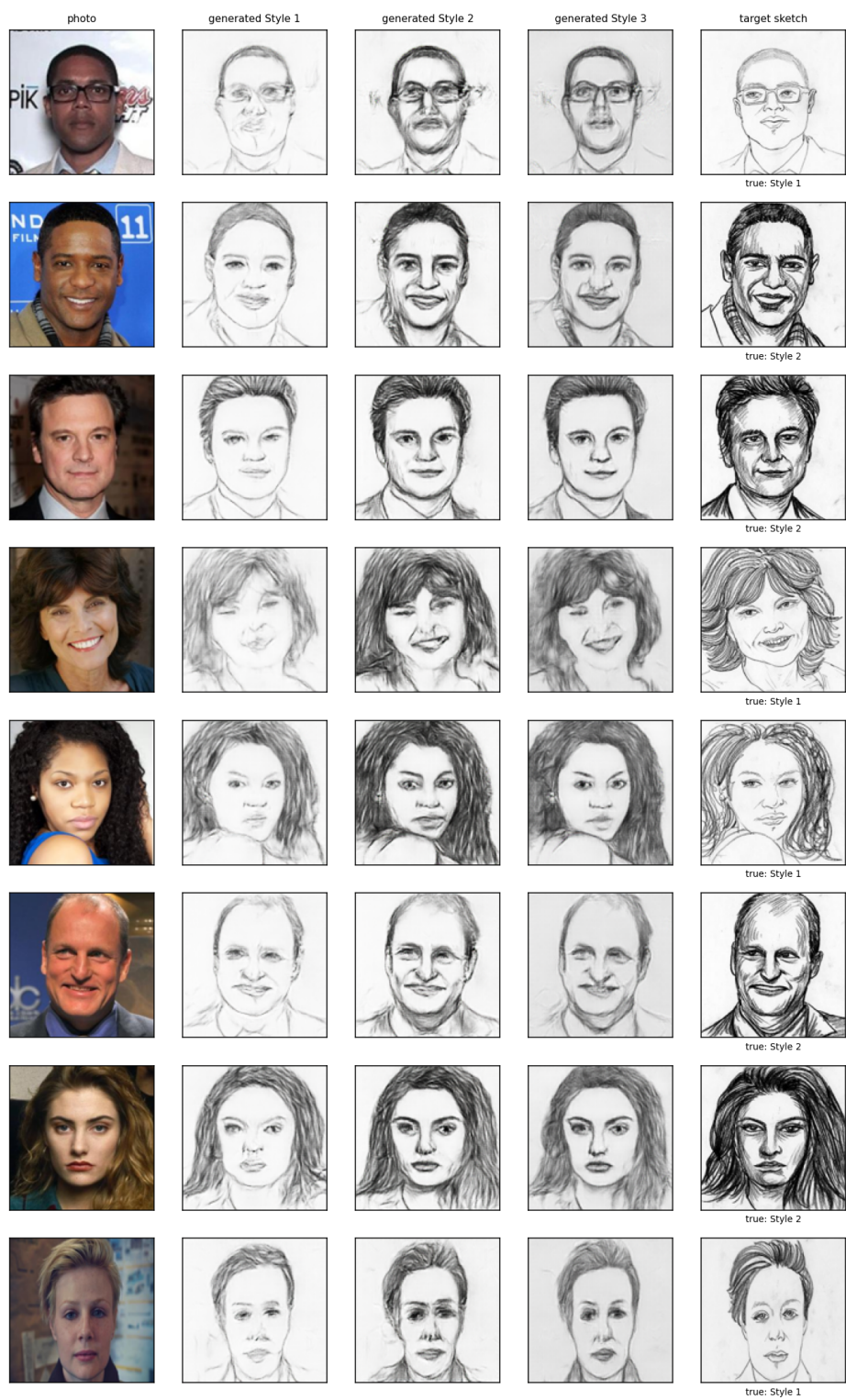


Fig. 7. Test photographs generated in each of the three styles next to the ground-truth sketch of their true style.

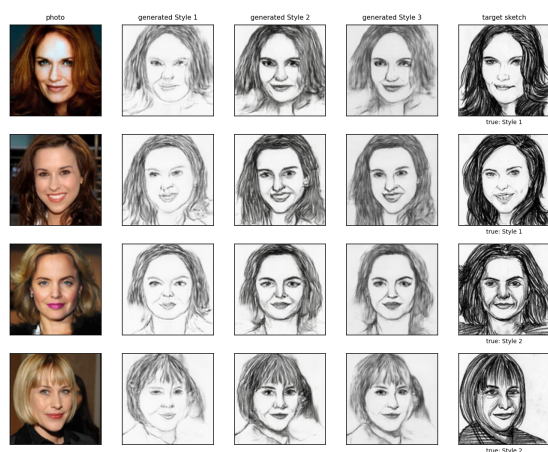


Fig. 8. Task 4 failure cases: the four test pairs with the largest L_1 error.

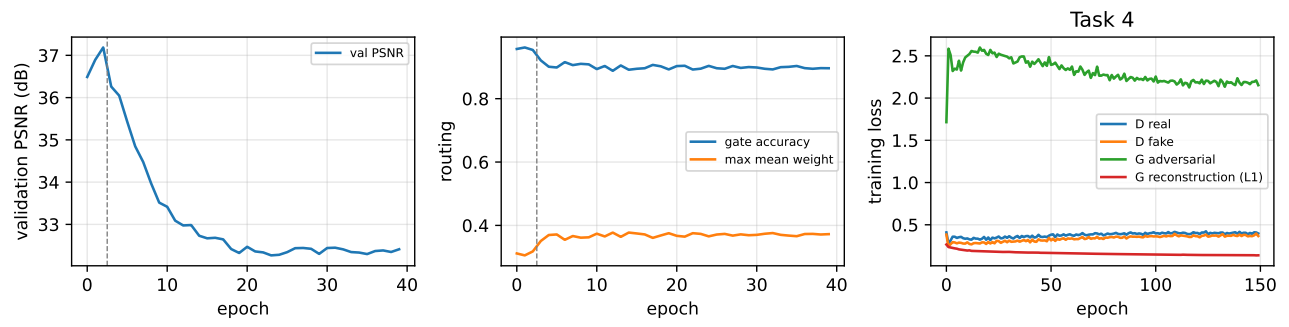


Fig. 9. Left: Task 3 validation PSNR (dashed line: start of joint fine-tuning). Middle: gate accuracy and the largest mean routing weight (collapse indicator). Right: Task 4 training losses (D-real, D-fake, G-adversarial, G-reconstruction).

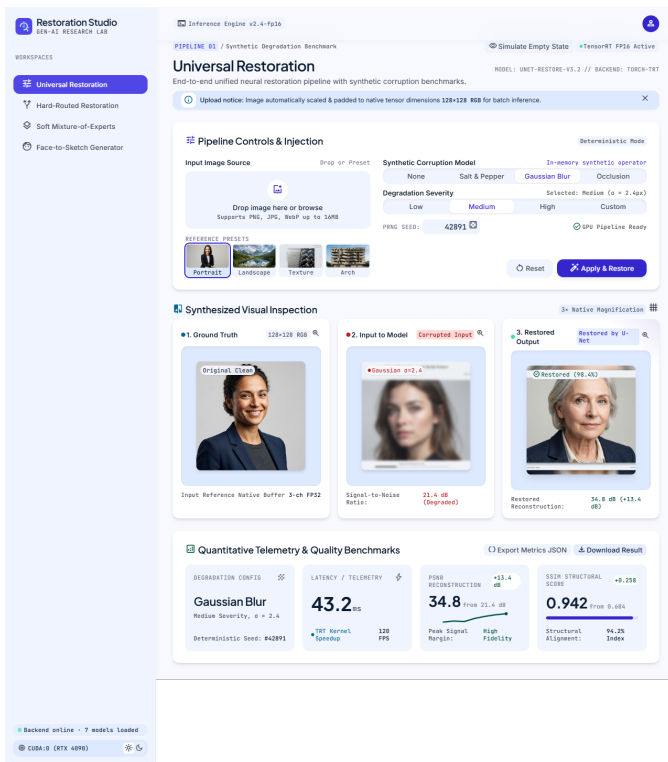


Fig. 10. Original Stitch design (first generation), Universal Restoration screen.

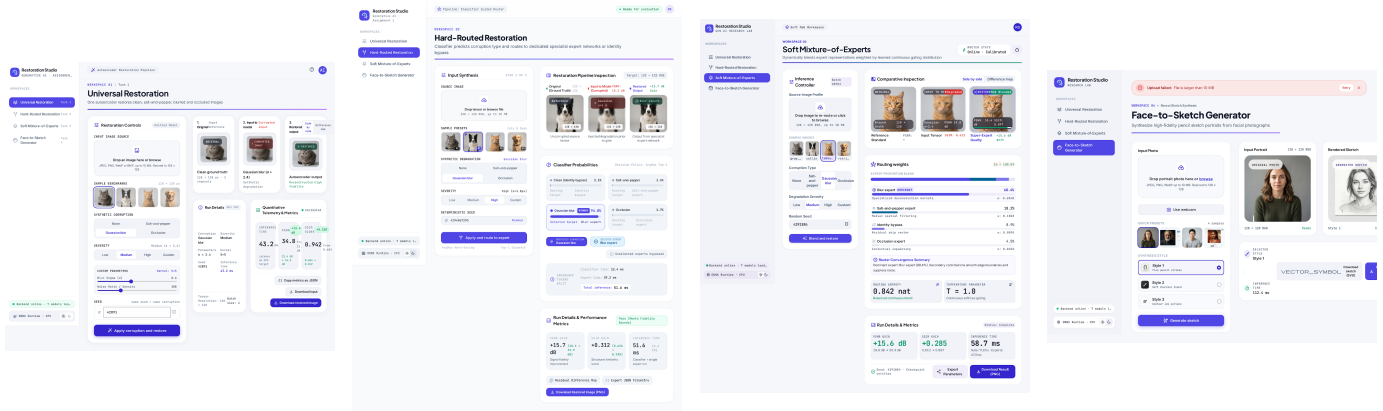


Fig. 11. Refined Stitch design: Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts and Face-to-Sketch Generator.